How Robust are LLM-Generated Library Imports? An Empirical Study using Stack Overflow

Jasmine Latendresse, SayedHassan Khatoonabadi, Emad Shihab

***Abstract*—Software libraries are central to the functionality, security, and maintainability of modern code. As developers increasingly turn to Large Language Models (LLMs) to assist with programming tasks, understanding how these models recommend libraries is essential. In this paper, we conduct an empirical study of six state-of-the-art LLMs, both proprietary and open-source, by prompting them to solve real-world Python problems sourced from Stack Overflow. We analyze the types of libraries they import, the characteristics of those libraries, and the extent to which the recommendations are usable out of the box. Our results show that LLMs predominantly favour third-party libraries over standard ones, and often recommend mature, popular, and permissively licensed dependencies. However, we also identify gaps in usability: 4.6% of the libraries could not be resolved automatically due to structural mismatches between import names and installable packages, and only three models (out of six) provided installation guidance. While the generated code is technically valid, the lack of contextual support places the burden of manually resolving dependencies on the user. Our findings offer actionable insights for both developers and researchers, and highlight opportunities to improve the reliability and usability of LLM-generated code in the context of software dependencies.**

***Index Terms*—Software engineering, software ecosystems, large language models.**

I. Introduction

MODERN software development heavily relies on open source libraries that provide reusable functionalities through well-defined modules, significantly reducing development time and effort [1]–[3]. While libraries can help speed up development tasks, they also introduce dependencies (interconnections between code components) that can lead to increased complexity and dependency management challenges [4]–[6].

One critical aspect of dependency management is library selection [7]. Choosing the right library impacts factors like code maintainability, performance, and security. Previous studies have explored how developers select libraries, and highlighted primarily ad-hoc processes based on past experiences, expert advice, and online resources [8], [9].

Decisions around dependency adoption are influenced by factors such as functionality, community support, and maintenance compatibility [7], [10], [11]. Previous studies reveal that maintenance tasks like updates and vulnerability migration are often hindered by risks of breaking changes [12]–[14] and complexity from transitive dependencies [15]–[18]. Automation and tooling (e.g., Dependabot [1]) have been introduced to alleviate these issues, but they frequently fall short because of alert fatigue, false positives, or lack of actionable context [19]–[23].

In parallel, the growing adoption of LLMs as programming assistants introduces new possibilities for addressing these challenges. LLMs are increasingly used to assist with code generation, and studies show their potential to enhance productivity through capabilities like code completion and search [24]–[26]. However, their impact on software dependencies remains underexplored. In particular, even when LLM-generated code is syntactically correct and the recommended libraries part of the generated code are meaningful, dependency-related issues may still arise at the level of usability and integration. For example, models may introduce unnecessary libraries, omit installation guidance, or recommend imports that don't cleanly map to installable packages. As these models become central in development workflows, it is therefore important to assess not only whether they generate working code, but also whether their library recommendations are easy to integrate in practice.

Our work aims to explore a specific aspect of LLM functionality: their ability to recommend software libraries during code generation. In this paper, we evaluate how six state-of-the-art LLMs recommend software libraries in a conversational coding-assistance setting by conducting an empirical study in which each model generates Python code for a set of real-world coding questions derived from Stack Overflow [27]. We focus specifically on Python and on the conversational question-answering setting, where developers ask programming questions in natural language and receive standalone code solutions. This setting allows us to study how LLMs recommend libraries when they are not constrained by an existing project context or import graph. Our analysis is guided by the following research questions:

RQ1 What types of software libraries are recommended by LLMs? Software libraries play a critical role in the security, performance, and maintainability of software projects. Thus, we examine the types of libraries recommended by LLMs and how frequently they rely on Python's standard library versus third-party. We found that third-party libraries constitute the majority of distinct imported names. Across all models evaluated, 54% of distinct names were classified as third-party, 41.4% as standard, and 4.6% as unresolved by our lookup.

J. Latendresse is with Concordia University, Montreal, Canada (email: jasmine.latendresse@mail.concordia.ca).

S. Khatoonabadi is with Concordia University, Montreal, Canada (email: sayedhassan.khatoonabadi@concordia.ca).

E. Shihab is with Concordia University, Montreal, Canada (email: emad.shihab@concordia.ca).

[1] https://github.com/dependabot

RQ2 What are the characteristics of third-party libraries recommended by LLMs? Third-party libraries introduce external dependencies that require separate installation and ongoing maintenance, which makes their quality and licenses critical in production contexts. To assess whether the libraries imported in LLM-generated code are correct and sustainable, we evaluate each third-party dependency along three key dimensions: popularity (e.g., GitHub stars, dependents), maintenance (e.g., age, update frequency), and licensing. Our findings show that LLMs tend to favour well-established and permissively licensed libraries that have high community adoption and low maintenance overhead.

RQ3 Why do some libraries recommended by LLMs not work out of the box? Some libraries recommended by LLMs do not work out of the box, which can be disruptive to the development workflow and negatively affect trust in AI-based programming assistants. In this RQ, we investigate the root causes of these unresolved imports by analyzing cases where the recommended library could not be mapped to a known standard or third-party package. We found that most of the unresolved imports stem from structural conventions in the Python programming language, where the import name differs from the package name. We also identified one case involving a module-level import error. While the code is valid, missing installation instructions limit usability. Only a few models, such as DeepSeek V3, included installation guidance.

Our findings show how effectively LLMs recommend software libraries and the factors that influence their behavior in this context during code generation. This contributes to a better understanding of model capabilities and limitations, and provides actionable insights for developers integrating LLMs into their workflows. We also highlight areas where models consistently produce reliable outputs, as well as scenarios where users should be cautious due to missing context or unresolved dependencies. Our work makes the following contributions:

- An empirical library characterization of LLM-generated Python code across six state-of-the-art models, quantifying how they select, import, and *use* libraries (including over-importing, unresolved and aliased imports, and the popularity, maintenance, and licensing characteristics of recommended dependencies).
- A set of actionable recommendations for developers using LLMs to streamline their workflows, including guidance for potential issues with LLM-suggested libraries.
- A dataset of LLM-generated Python code from six state-of-the-art LLMs that enables a direct comparison of code outputs and library selection practices, available online[2].

Paper Organization. The rest of the paper is organized as follows. Section II describes our dataset and methodology. Sections III, IV, V present the findings of our three research questions. Section VI discusses the implications of our findings along with our recommendations. Section VII discusses related work. Section VIII outlines the threats to the validity of our study. Finally, Section IX concludes this paper.

[2]https://zenodo.org/records/18022086

II. Study Design

In this section, we describe the steps to curate the dataset used in our study and the experimental setup.

A. Dataset

Figure 1 presents an overview of our approach for curating the dataset used in this study and analyzing the dependencies generated by LLMs. We use the StackEval benchmark suite curated by Shah et al. [28], which contains question–answer pairs sourced from Stack Overflow. The benchmark is designed to evaluate LLM performance on coding tasks and includes 925 questions sampled between January 2018 and September 2023. Each question includes a title, body, at least one upvote, and an accepted answer that also received one upvote.

We selected this dataset because Stack Overflow questions reflect real-world programming scenarios where developers seek help with writing, fixing, or improving code. These questions may involve software libraries either explicitly (e.g., *Difference between numpy power and ** for certain values*) or implicitly through the broader coding task. This allows us to examine how LLMs introduce and use libraries in a practical question-answering setting, rather than through tasks artificially constructed around dependency selection. In addition, StackEval matches a common interaction mode for LLM assistants, namely chat-style programming questions, and each item includes an accepted human-written answer that we use as a reference point in our ground-truth comparison (Section VI). Because accepted answers vary in whether they contain imports, we use the full Python subset when analyzing whether and how models introduce libraries in their own completions. When comparing model-generated imports against accepted answers, however, we restrict that comparison to the subset of questions whose accepted answers contain imports.

At the same time, this dataset is not a benchmark in which every task explicitly asks the model to select a library. In many questions, library choice is secondary to the broader programming problem. We therefore do not treat StackEval as a comprehensive benchmark of library selection in isolation. Instead, we use it to study library-related behavior in a realistic conversational coding-assistance setting, where dependency decisions may arise implicitly but still affect usability, installation effort, and integration burden.

More broadly, we acknowledge that benchmark-based LLM evaluations have known limitations, including possible contamination, benchmark-specific overfitting, and uncertainty about whether strong benchmark performance reflects true generalization. Prior work has raised these concerns both for LLM benchmarks in general and for code-generation benchmarks specifically [29], [30]. Although our methodology could be adapted to such benchmarks to study other usage modes, in this work we focus specifically on StackEval's Stack Overflow-style Q&A setting.

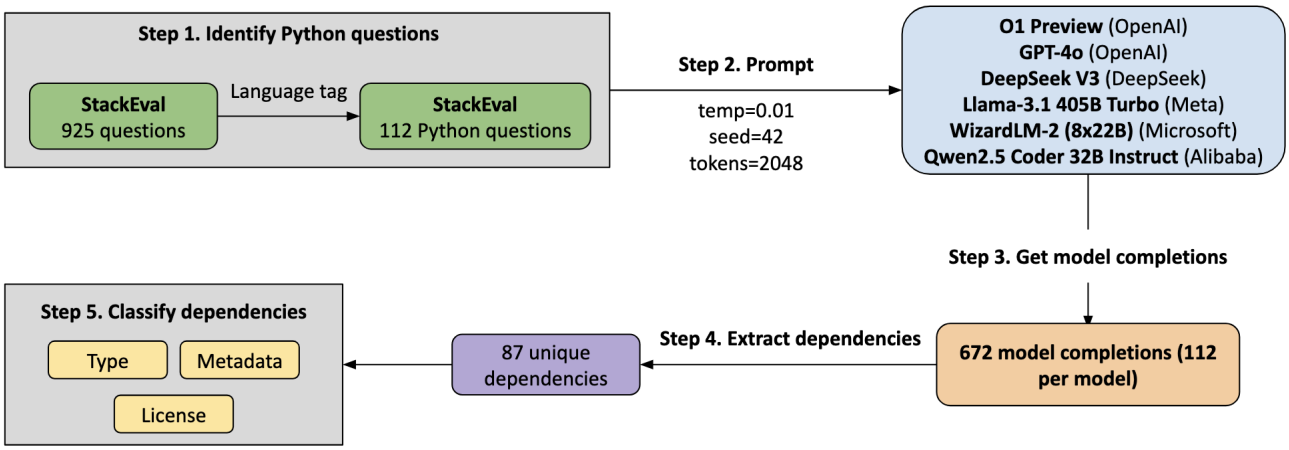


Fig. 1: An overview of our approach for filtering and curating our dataset.

We filtered StackEval to retain Python-related questions using the language tags. Python was chosen due to its popularity, rich ecosystem of third-party libraries, and the strong performance of LLMs in generating and understanding Python code [31]–[34]. We deliberately restrict our analysis to a single language and ecosystem for methodological reasons. This allows us to isolate model behavior more clearly, rather than conflating it with differences across language ecosystems, package registries, and import conventions. Including another ecosystem such as npm would require separate parsers, heuristics, and metrics, and would make it difficult to disentangle differences in model behavior from differences in ecosystem structure, package managers, and import conventions. We therefore treat Python as a representative ecosystem and leave cross-language comparisons to future work. Thus, our final dataset consists of 112 Python programming questions.

To better understand the kinds of problems represented in this subset, we inspected the question-level metadata provided by StackEval (task type and difficulty). The 112 Python questions span four task types: debugging (49 question, 43.8%), implementation (43, 38.4%), conceptual understanding (14, 12.5%), and optimization (6, 5.4%). Most questions are labelled as intermediate (87, 77.7%), with the remainder split between beginner (16, 14.3%), and advanced (9, 8%). This yields 11 distinct (type, level) combinations). Question texts are also non-trivial in length, and range from 143 to 3,845 characters (median 790) and from 5 to 116 lines (median 32). Overall, while the Python subset is small in comparison to the overall set, it covers a diverse mix of realistic Python debugging, implementation, and conceptual tasks at predominantly intermediate complexity.

B. Experiment Setup

We evaluate six state-of-the-art LLMs, including two proprietary models and four open-source models. The proprietary models are O1 Preview [35] and GPT-4o [36], both developed by OpenAI. The open-source models include DeepSeek V3 [37] from DeepSeek AI, Llama 3.1 405B Turbo [38] from Meta, WizardLM-2 8x22B [39] from Microsoft, and Qwen 2.5 Coder 32B Instruct [40] from Alibaba. Table I provides an overview of their key characteristics and capabilities. All model inference was performed through the TogetherAI platform. This applies to both the proprietary and open-weight models evaluated in our study.

To collect model completions, we follow the approach of Shah et al. [28]. Each model is prompted with the selected 112 Python questions from StackEval using only the question title and body, without additional instructions or examples. In line with Shah et al., we intentionally avoid any further prompt engineering beyond what is present in the original Stack Overflow question: we do not prepend meta-instructions such as “include installation instructions“ or “explain your imports“. This setup reflects realistic usage, where developers typically pose natural language questions without carefully crafted prompts [41], and allows us to evaluate model behavior in conditions that are minimally guided and standardized across models. The parameter configuration has a near-deterministic temperature (0.01), a fixed seed (42), and outputs limited to 2,048 tokens. These settings follow the configuration used in the original StackEval benchmark [28], and are chosen to minimize the randomness of the output and ensure reproducibility. While higher temperatures and multiple samples are sometimes used to explore the diversity of generated code, our goal in this study is to characterize each model’s most likely completion for a given question, analogous to the single answer a developer would typically inspect. We therefore generate one low-temperature completion per question and leave the analysis of full output distributions under higher temperatures as complementary future work.

We emphasize that this setup is intended to characterize model behavior in a conversational coding-assistance workflow, not to measure the maximum achievable performance of tool-augmented or agentic LLM systems. In particular, we do not provide execution feedback, package-resolution tools, linters, or iterative repair loops. We chose this design to isolate the model’s own library recommendation behavior under a minimally engineered prompt, and because single-turn question-answering remains a common way developers interact with LLM assistants in practice. Evaluating how

TABLE I: Overview of evaluated LLMs

Model	Provider	Nb. Parameters	Publication Year	Training Cutoff	Capabilities
DeepSeek V3	DeepSeek AI	685B	December 2024	July 2024	Mixture-of-experts, general-purpose, code, reasoning.
GPT-4o	OpenAI	∼1.8T	April 2024	December 2023	Multimodal, general purpose, code, reasoning.
Llama 3.1	Meta	405B	July 2024	December 2023	Instruction-tuned, general purpose, text, code.
O1 Preview	OpenAI	∼175B	September 2024	October 2023	Reasoning-optimized.
Qwen2.5	Alibaba	32B	September 2024	March 2024	Instruction-tuned, strong support for code.
WizardLM-2	Microsoft	141B	April 2024	October 2023	Mixture-of-experts, instruction-tuned, code.

the same dependency-related behaviors change under agentic scaffolding is an important direction for future work.

After generating responses, we first extract code blocks from each completion using a regular expression over fenced code (e.g., ```` ```python ```` blocks). We then parse each code block with Python's `ast` module and identify all `import` and `from X import Y` statements and normalize them to top-level names (e.g., `pandas.core.frame` → `pandas`). This allows us to focus on high-level usage patterns of libraries rather than individual submodules. Across the 112 questions and all models, this process yields 87 unique libraries. In our analysis, we treat each library as an individual case, regardless of how many times it appears across questions or models. For per-model analysis, the same library may appear in multiple completions, whereas for aggregated analysis we focus on the set of unique libraries recommended across all completions. Finally, we classify each library by type (standard, third-party, unresolved) and collect metadata such as its ecosystem and license; this classification procedure is detailed in RQ1.

III. RQ1: What types of software libraries are recommended by LLMs?

Motivation. As developers increasingly turn to AI-based programming assistants to streamline their development workflows, the choice of libraries included in AI-generated code becomes critical. These libraries directly impact the resulting code's performance, security, and functionality [4], [42], [43]. This research question aims to examine the types of libraries included in LLM-generated code and the distribution of distinct imported names across standard, third-party, and unresolved categories. Understanding this helps evaluate the reliability and integration complexity of LLM-generated library imports in real-world development settings.

Approach. We classify each imported library as one of the following: (1) *standard*, meaning it is bundled with the default Python installation, (2) *third-party*, meaning it is installable via the Python Package Index (PyPI), or (3) *unresolved*, meaning its extracted import name is not matched to either category by our automated lookup. To perform this classification, we use the `stdlib-list` package to identify standard libraries (for Python 3) and query the PyPI API using the remaining import names. Names not found in either lookup are labeled as *unresolved*. This label denotes a limitation of direct name-based lookup, rather than evidence of an invalid import or a hallucinated library. In particular, valid imports such as `cv2`, `dateutil`, and `yaml` can be labeled as unresolved because their import names differ from their installable package names. We manually examine these cases in RQ3.

Results. Table II presents the distribution of dependency types among unique imported library names, with percentages calculated over each row's total. These values describe library diversity rather than import frequency or the percentage of completions containing imports. We identified a total of 87 distinct dependencies of which 47 (54.0%) were *third-party* libraries, and 36 (41.4%) were *standard* libraries. Thus, third-party libraries account for the largest share of distinct imported names. Moreover, four (4.6%) of the libraries were classified as *unresolved*, indicating these libraries are neither identified as *standard* nor *third-party*. We further investigate this category and its implications in RQ3.

TABLE II: Classification of distinct imported library names in LLM-generated code.

Model	Std.	3rd-Party	Unresolved	Total
All models	36 (41.4%)	47 (54%)	4 (4.6%)	87
DeepSeek V3	24 (42.1%)	30 (52.6%)	3 (5.3%)	57
GPT-4o	26 (40.6%)	35 (54.7%)	3 (4.7%)	64
Llama 3.1	22 (40.7%)	28 (51.9%)	4 (7.4%)	54
O1 Preview	18 (39.1%)	26 (56.5%)	2 (4.3%)	46
Qwen2.5	21 (36.2%)	34 (58.6%)	3 (5.2%)	58
WizardLM-2	19 (38.8%)	28 (57.1%)	2 (4.1%)	49

Looking at the results per model, we observe a consistent pattern across all six LLMs: third-party libraries account for the largest proportion of distinct imported names. GPT-4o recommends the most unique libraries overall (64), with *standard* libraries accounting for 40.6%, third-party accounting for 54.7%, and three distinct unresolved names. Llama 3.1 has the highest proportion of unresolved names (four, or 7.4%), compared with two or three names (4.1–5.3%) for the other models. This small variation is not substantial enough to indicate systematic differences between models. Nonetheless, its proportions of *standard* and *third-party* imports remain comparable to other models. O1 Preview has the lowest total number of distinct dependencies (46), but maintains a similar overall trend, with a majority of third-party recommendations (56.5%). Qwen2.5 has the highest proportion of *third-party* dependencies (58.6%), which could have implications for integration complexity and maintenance. WizardLM-2 has the smallest proportion of unresolved (4.1%), but also shows a high proportion of distinct third-party names (57.1%). Overall, we can see that all models tend to use third-party libraries more often than *standard* with varying degrees (e.g., Qwen2.5, WizardLM-2).

Our analysis shows third-party libraries account for 51.9–58.6% of distinct imported names across models. Out of 87 distinct libraries identified across all completions, 54.0% were third-party, 41.4% were standard, and 4.6% were unresolved. GPT-4o generated the broadest set of libraries, while Llama 3.1 has the highest proportion of unresolved.

IV. RQ2: What are the characteristics of third-party libraries recommended by LLMs?

Motivation. Unlike standard libraries, third-party libraries introduce external dependencies that must be installed and maintained separately. Their characteristics, such as popularity, maintenance activity, and licensing, are known to significantly affect the long-term stability and legal compliance of software projects [10]. Thus, in this research question, we aim to evaluate whether LLMs tend to recommend third-party libraries that are mature, reliable, and safe for production.

Approach. We analyze each third-party library along three key dimensions based on prior work [11], [42]: popularity, maintenance, and licensing. For popularity, we examine the number of GitHub forks, stars, and dependent projects. For maintenance, we look at the number of dependencies, age (calculated from the library's first release date to the metadata collection date (September 2025)), version release frequency (releases per month from first release to our September 2025 snapshot, using libraries.io version timestamps), and SourceRank [3] score (provided by libraries.io). Finally, we assess licensing information by assigning SPDX license identifiers and grouping them into categories such as permissive, copyleft, and weak copyleft.

Results. In this section, we analyze the characteristics of third-party libraries recommended by LLMs. As mentioned in RQ1, across all models, we observe 47 distinct packages and 343 third-party import occurrences. The distribution is skewed towards widely used packages: the five most frequent libraries (`pandas`, `numpy`, `sklearn`, `torch`, and `django`) account for 51.9% of all third-party import occurrences, and the top ten libraries (adding `matplotlib`, `boto3`, `xgboost`, `imblearn`, and `pyspark`) account for 63.8%. The remaining 36.2% of occurrences are spread across 36 additional libraries that each appear between one and six times. We provide the complete list of libraries and their frequencies in our replication package to make this distribution fully transparent. Thus, the results discussed below should be interpreted as indicative of those libraries rather than exhaustive characterizations of the entire Python ecosystem. Table III presents the median values of key dependency characteristics for each model as well at the aggregated results. Below, we discuss the results in more detail.

Popularity. The popularity metrics (forks, stars, and dependents) show how widely adopted and trusted a library is within the Python ecosystem. They reflect developer interest (e.g., stars), collaborative engagement (e.g., forks), and real-world usage (e.g., dependents). The aggregated medians across models show that LLMs generally recommend libraries that are well-established and commonly used, with a median number of forks of 1,473, 6,962 stars, and 1,140 dependents.

At the per-model level, we observed that the median number of stars ranges from 5,471 to 8,528. This indicates that all models recommend libraries with a notable level of visibility and level of adoption. As shown in Table III, some models fall toward the high end of this range (e.g., Qwen2.5 and Llama 3.1), and some fall toward the lower end (e.g., WizardLM-2 and DeepSeek V3), suggesting that they might recommend more specialized or context-specific libraries. Other models have popularity metrics that are close to the overall medians (e.g., GPT-4o on all popularity metrics and O1 Preview on forks and dependents), which suggests that their recommendations are still broadly visible and adopted, but not skewed towards highly popular or niche options.

Maintenance. In terms of maintenance, the results show that most models recommend libraries that are mature and self-contained. The median number of dependencies across all models is low (1.0) and never exceeds 3.0 at the per-model level. This indicates that the recommended libraries are lightweight in terms of transitive dependencies, reducing integration risk and potential for version conflicts in downstream projects [10].

The SourceRank metric, which reflects a library's overall quality and development activity, is relatively consistent across models, ranging from 22 to 24, with an overall median of 23 (out of a maximum of 30). This indicates that LLMs tend to select libraries that are well-maintained and well-regarded in the community. The age of recommended libraries is consistently high, with median values across models above 7 years, which suggests LLMs tend to recommend stable, long-standing libraries over newer ones. From a developer's perspective, this behaviour may promote trust and ensure compatibility, especially in production environments where stability and maturity are important.

Version release frequency remains below one release per month for all models, with medians ranging from 0.58 to 0.77 (˜17 to 23 days). These values imply that most libraries receive regular but not overly frequent updates, which is often preferred by developers who want to avoid breaking changes, frequent manual interventions, or unexpected regressions [44]. The observed range for version frequency does not appear to correlate directly with the training cutoff dates. For instance, O1 Preview (cutoff: October 2023) and Qwen2.5 (cutoff: March 2024) both recommend libraries with identical median version frequency per month (0.77). This suggests that other factors like training data composition, domain coverage, or fine-tuning strategies may have a greater influence on the kinds of libraries selected by each model.

License. We next examine the licenses associated with the third-party libraries recommended by each model. Of the 181 license instances across all models, 164 (91%) fall into the permissive category, primarily MIT, BSD, and Apache-2.0 (we consolidate BSD-2-clause and BSD-3-clause into a single BSD category). Permissive licenses impose minimal obligations such as attribution and allow use in proprietary software, so

[3]https://github.com/librariesio/libraries.io/issues/1916

TABLE III: Median values of recommended third-party library characteristics.

Category	Metric	All Models	DeepSeek V3	GPT-4o	Llama 3.1	O1 Preview	Qwen2.5	WizardLM-2
Popularity	Forks	1,473	1,292	1,473	1,682	1,682	1,430	1,292
	Stars	6,962	5,607	6,962	8,528	8,073	8,073	5,471
	Dependents	1,140	1,068	1,114	1,068	1,287	2,080	1,287
Maintenance	Dependencies	1.0	0.5	2.0	2.5	0.0	1.5	1.5
	SourceRank	23	22.5	23	22	22.5	24	22
	Age (months)	95	94.9	97.7	92.9	87.3	99.5	104.1
	Version Frequency	0.64	0.62	0.58	0.62	0.77	0.77	0.77

they are generally preferred in industrial settings. In contrast, copyleft licenses (e.g., GPL family) require derivative works to remain open source, while weak-copyleft licenses (e.g., PSF-2.0, MPL-2.0) sit in between by allowing proprietary use but imposing conditions on modifications to the licensed component. DeepSeek V3 is the only model that recommended a copyleft-licensed library (`certifi`), and each model recommended a weak-copyleft PSF-2.0 library (`matplotlib`).

However, each model consistently recommends the same three libraries for which the license is not explicitly declared on PyPI (*unresolved*): `myapp`, `sklearn`, and `imblearn`. For `myapp`, the documentation indicates that it is a test library [4], which could explain why no license is explicitly declared. We observed that the models use this name as a placeholder instead of referring to the actual library.

In the case of `sklearn`, the corresponding PyPI page indicates that the library is deprecated [5], which led us to classify it as having no declared license. However, we found that `sklearn` is an import alias for the actively maintained `scikit-learn` library. This means that while developers install the library using the name `scikit-learn`, it is imported in Python as `sklearn`. As a result, querying `sklearn` directly on PyPI leads to a deprecated library. In reality, `scikit-learn` is distributed under a permissive type of license (BSD). We discuss the implications of such aliasing further in RQ3. Finally, for `imblearn`, the PyPI documentation recommends using `imbalanced-learn` instead [6], yet no explicit deprecation notice is shown and unlike `scikit-learn`, we found no evidence of aliasing for this case in our dataset. Thus, the results show that LLMs recommend libraries that are widely permissive, which is ideal in policy-sensitive production settings, but there are a few cases where the license can be more restrictive (e.g., copyleft) or not explicitly declared, which can lead to potential legal challenges. Practitioners should actively verify the license of the libraries recommended by LLMs they integrate into their codebases.

[4]https://pypi.org/project/myapp/
[5]https://pypi.org/project/sklearn/
[6]https://pypi.org/project/imblearn/

Our findings show that LLMs predominantly generate dependencies on third-party libraries that are popular, well-established, and permissively licensed. While these characteristics are generally reassuring, they also indicate that LLM-generated code often expands the external dependency surface, making downstream usability and integration considerations important. Across all models, the recommended libraries are stable (median version frequency of 0.64), have minimal transitive dependencies (median of 1 dependency), and are well-regarded by the community (median SourceRank of 23).

V. RQ3: Why do some libraries recommended by large language models not work out of the box?

Motivation. In RQ1, we observed that 4.6% of distinct imported names in LLM-generated code were unresolved by our automated name-based lookup. This does not necessarily indicate invalid imports or hallucinated libraries: valid Python import names can differ from their installable package names. In a baseline conversational workflow, identifying the corresponding packages may require additional user effort, particularly when installation guidance is absent. Such dependency-management friction may affect trust in the output [33]. Thus, RQ3 examines why these import names remain unmatched by our lookup and whether the accompanying responses provide installation guidance. We distinguish conventional import–package name mismatches from module-level import issues, interpreting their practical implications in terms of dependency resolution rather than treating all unresolved cases as evidence of incorrect code.

Approach. To understand why some libraries recommended by LLMs do not work out of the box, we analyzed each case identified as *unresolved* in RQ1 (i.e., neither *third-party* nor *standard*). Specifically, we qualitatively examined the context in which each library was recommended by analyzing the associated Stack Overflow question used in the prompt, including its body, code snippets, and accepted answer. This process allowed us to infer whether the recommendation stemmed from a misunderstanding of naming conventions, structural issues in the import statement, or other contextual factors. Based on this, we developed a set of mutually exclusive categories that characterize the underlying reasons why certain libraries in LLM-generated code fail to resolve correctly. Since this categorization relied on concrete, factual properties, the first author performed the initial labeling, and the results were

reviewed and validated by the second author. This led to the identification of two primary categories:

- **Alias**: The extracted library name corresponds to an alias used in Python import statements, rather than the actual name of the library used for installation. For example, `sklearn` is used as an alias for the `scikit-learn` library. While the model's suggestion is functionally correct, alias-based imports may lead to errors during dependency resolution.
- **Module**: The extracted name refers to an internal module or subcomponent within a third-party library. These cases occur when the model suggests imports from a specific file using non-standard or path-based import patterns. These cases also happen with implicit imports, which is when a module is imported directly without referencing the parent library. Since the name used in the import does not match the actual installable library, it can lead to a failure in dependency resolution if the parent library is not installed.

Results. We first present the underlying causes of unresolved imports, and then discuss their practical consequences in terms of installation guidance and user burden. Table IV shows the *unresolved* libraries identified in RQ1, categorized into the two underlying causes discussed above: *alias* and *module*. The *alias* category accounts for most of the *unresolved* cases across all six models, with a total of 24 occurrences (3 unique). The most frequent alias is `cv2` used for the `opencv-python` library, which appears in all models. Similarly, `dateutil` is used as an alias for `python-dateutil`, also appearing in all models. The last alias, `yaml`, used for the library `PyYAML`, appears in half of the models (GPT-4o, Llama 3.1, and Qwen2.5).

These imports are functionally correct, but can lead to confusion since the alias used in the import statement does not match the actual installable library name. To further investigate the practical consequences of this, we manually reviewed the responses generated by the models for each of the 24 alias cases. We found that only 5 of these completions included an installation step (e.g., `pip install opencv-python`), which occurred in outputs from DeepSeek V3, GPT-4o, and Qwen2.5. This indicates that models may often assume implicit knowledge about the correct library to install and places the burden on developers to manually infer the library before they execute the code. This is especially important in educational contexts, where users may lack the knowledge to resolve missing dependencies.

In contrast to *alias* cases, we identified a single instance categorized as *module*. This occurred in a response generated by Llama 3.1, where the model produced the statement `from client import EmailageClient`, which resulted in the extraction of `client` as the top-level dependency. However, `client` is not a standalone installable library; it is a submodule nested within the `emailage` library. Thus, the import statement is not incorrect in itself, but attempting to install `client` on its own would fail. This is a challenge that occurs with implicit imports: the module is imported without an explicit reference to its parent library. In this particular case, Llama 3.1 did not provide additional installation steps in the completion, leaving it to the user to manually infer the library to install. It could be argued that the user might possess the necessary knowledge given that they formulated the original question, but this assumption is problematic. Similar to the alias cases, it places the burden of dependency resolution on the user and undermines the practical reliability of LLMs in real-world settings.

Practical implications and model differences. Although all models produce some *unresolved* libraries, their behavior varies slightly. For instance, GPT-4o, Qwen2.5, and Llama 3.1 tend to rely more heavily on alias imports. However, these aliases (e.g., `cv2`, `dateutil`, `yaml`) are widely adopted and well-established libraries. Interestingly, only these three models used the `yaml` alias, despite all models receiving the same input, which suggests that they may have a more accurate understanding of the `PyYAML` library to recommend it for the same use case. In contrast, WizardLM-2 and O1 Preview produced the fewest *unresolved*, which could suggest they are more conservative about importing libraries. This is reflected in RQ1, where these models produced the fewest unique imports overall.

DeepSeek V3, which produced a moderate number of aliases (4), was one of the few models that included installation guidance (with GPT-4o and Qwen2.5), indicating that the model can both produce correct imports and correctly map aliases to their installable libraries. This may indicate that models which include installation guidance in their completion anticipate potential friction during code execution and proactively address it. While this does not guarantee robustness, it suggests a model-level difference in how much contextual support is embedded model's output, a feature that enhances usability and trust.

Our results show that all models are capable of generating technically correct code, but some models may place a greater burden on the user to interpret and resolve dependencies, especially when explicit installation instructions are omitted from the output.

Our findings show that the majority of unresolved library references stem from aliasing, where the import statement refers to a library name that differs from its installable package name. We also identified one case involving an implicit module import, in which a submodule of a library is referenced without its parent library. We conclude that while the code produced is syntactically correct, it lacks sufficient context to ensure seamless execution and places a burden on users to resolve dependencies manually. Only a few models, such as DeepSeek V3, offered installation guidance, highlighting key differences in practical usability across models.

VI. Discussion

In this section, we discuss the implications of our work and propose recommendations for practitioners and researchers in

TABLE IV: Occurrences of alias and module cases per model.

Model	# Alias Occ.	Alias instances	# Module Occ.	Module instances
DeepSeek V3	4	`cv2` (1), `dateutil` (3)	0	–
GPT-4o	5	`cv2` (2), `dateutil` (2), `yaml` (1)	0	–
Llama 3.1	5	`cv2` (1), `dateutil` (2), `yaml` (1)	1	`client` (1)
O1 Preview	3	`cv2` (1), `dateutil` (2)	0	–
Qwen2.5	5	`cv2` (1), `dateutil` (3), `yaml` (1)	0	–
WizardLM-2	2	`cv2` (1), `dateutil` (1)	0	–
Total	24		1	

the context of LLMs and dependencies.

A. How do LLMs perform on unseen questions?

A central threat to validity in studies using LLM-generated code is the possibility of data leakage (i.e., models having been exposed to the same or similar questions during training). To address this, we repeated our analysis on StackUnseen, a separate dataset specifically curated to reduce this risk [28]. This dataset complements StackEval by focusing on more recent Stack Overflow questions (as of October 2024), which are less likely to have been present in the models' training corpora. From the 494 questions in StackUnseen, we extracted 71 Python-related questions and applied the same methodology as used in the StackEval analysis, which yielded a set of 66 unique libraries. Among these 66 libraries, 38 are third-party, 25 are standard-library modules, and 3 are unresolved. As a reminder for comparison, StackEval contains 87 distinct libraries (47 third-party, 36 standard, and 4 unresolved).

Thus, in this analysis, we examine whether the characteristics of the third-party libraries differ between StackEval and StackUnseen. In particular, we ask whether model behavior changes when faced with unseen questions, as reflected in the characteristics of the libraries they recommend. A substantial shift in these characteristics could indicate a reliance on previously seen data and would raise concerns about the generalizability of our results.

To achieve this, we compared StackEval and StackUnseen at the *per-model* level. For each model, we considered unique third-party libraries (deduplicated by `(model, dep)`) and compared the distributions of library characteristics between the two datasets using Mann–Whitney U tests. We repeated this test for each characteristic (number of stars, forks, dependents, dependencies, SourceRank, age in months, and version frequency) and applied a Holm correction for multiple comparisons across all model–metric combinations.

Across all models and characteristics, we found no statistically significant differences between StackEval and StackUnseen (all Holm-corrected $p \geq 0.05$). While individual medians can vary slightly between the two datasets, these shifts are small and not statistically reliable after correction. In other words, for a given model, the libraries recommended on unseen questions exhibit similar popularity and maintainability characteristics as those recommended on historical questions. However, aggregate characteristics do not capture how often recommendations fail to work out of the box (i.e., unresolved). Thus, in addition to evaluating the characteristics of recommended libraries (RQ2), we examined the subset of libraries that were identified as *unresolved* (i.e., not clearly identifiable as *standard* or *third-party*). These represent edge cases where model recommendations may not work out of the box and require manual resolution from the user. As seen in RQ3, in StackEval, we identified 24 such cases (4.3% of the recommendations), primarily due to aliasing and module-level imports.

In contrast, StackUnseen yielded only 4 *unresolved* cases (1.2%). Aliasing accounts for 50% of these cases (`browsermobproxy`, `seleniumrequests`), while placeholder usage accounts for the other 50% (`some_module`). In this context, a placeholder refers to a stand-in or generic example used by the model that is meant to be replaced by the user with a real library. This reduction in `unresolved` cases may be due to models defaulting to more popular or generic imports when faced with truly unseen data. It could also be partially influenced by the smaller sample size in StackUnseen, which reduced the likelihood of observing edge cases. Therefore, it is not possible to draw firm conclusions about the influence of unseen questions on this outcome.

Our findings have practical implications for developers relying on LLM-based code generation tools. First, the consistency in library characteristics between StackEval and StackUnseen shows models exhibit relatively robust behavior in their library recommendations, even when answering questions unlikely to have been seen during training. This reinforces the reliability of our findings and supports the use of StackEval as a benchmark for future evaluations. Second, although rare, the presence of *unresolved* cases, even in unseen data, highlights the need for caution when integrating LLM-generated code into software projects. Developers should be aware that LLMs may suggest libraries that do not install cleanly or require specific domain knowledge to resolve. Incorporating mechanisms that prompt LLMs to clarify or justify their library usage could improve usability in real-world development workflows.

B. Are LLMs Importing Libraries They Do Not Use?

To better understand whether the imported libraries are actually used by the generated code, we perform a static analysis over all model completions. Building on the import-extraction pipeline described in Section II-B, we take each completion, inspect the AST to locate import statements, and then check whether the bound name is ever read in the body of the generated code block (e.g., used in an attribute access such as `pandas.DataFrame` or as a bare function or class imported via `from X import f`). We then classify each

import as used or unused (i.e., dead import) and aggregate the results per model.

Across all models, the median number of imports per completion is two, indicating that the generated snippets are not heavy in terms of declared dependencies. Table V summarizes how often these imports remain unused. As shown in the column ”All imports”, the majority of the models leave around 5-7% of their imports unused, with WizardLM-2 at the highest proportion with 12%. These results suggest that the majority of imported names are used in the corresponding generated code blocks.

Table V also breaks down unused imports by dependency type (standard, third-party, unresolved). Unused standard-library imports range, for the majority, between 2 and 6%, and even 0 in the case of O1 Preview. Third-party imports have higher dead-import rates, with values between 4 and 15% depending on the model. For example, WizardLM-2 leaves about 14.5% of its third-party imports unused, compared to 5.6% for standard-library imports. This indicates that, in general, when models ”over-import”, they tend to do so with external libraries rather than standard libraries, which is more concerning from a dependency-management perspective. Finally, the ”unresolved” column corresponds to libraries that could not be mapped to standard or third-party libraries. This group shows higher unused ratios for some models (e.g., O1 Preview and Llama 3.1) but represent a small subset of imports, so these ratios should be interpreted with caution.

Our analysis correctly marks the top-level `import pymongo` as unused, since the body of the code only ever refers to the exception class imported via `from pymongo.errors import BulkWriteError`. From a runtime perspective, the package is loaded via the second import, so the first line is redundant. This pattern is consistent with several generation behaviours: the model ”stitches together” a generic import template (`import pymongo`) with a more specific error-handling idiom (`from pymongo.errors import BulkWriteError`) [45]. This pattern mirrors common tutorial code that imports both the parent package and one of its submodules, even when only the submodule is needed. *Our StackEval vs. StackUnseen results in Section VI-A suggest that this behaviour is better explained by models learning common import patterns from many similar examples in their training data than by direct memorization of a specific historical solution, although some overlap with training data cannot be entirely ruled out.* Thus, when lacking project-wide context, the model tends to err on the side of importing more than necessary rather than risk a missing symbol [46]. In all of these cases, the resulting snippet is syntactically correct, but it leaves behind imports that do not contribute to the behavior of the solution. This reveals a tendency for models towards ”template-like” code generation that can inflate the dependency surface, which is a form of dependency bloat that practitioners and tool builders should monitor or mitigate.

Overall, our analysis shows that the import statistics are not dominated by unused imports. Most imported libraries are actually referenced in the generated code, which indicates that models are generally not adding dependencies arbitrarily.

TABLE V: Percentage of import occurrences that are unused, per model, overall and within each dependency type.

Model	All imports	Std.	Third-party	Unresolved
DeepSeek V3	6.0	5.4	4.5	17.6
GPT-4o	5.7	1.9	5.4	23.1
Llama 3.1	6.7	2.0	6.0	28.6
O1 Preview	5.7	0.0	1.6	41.7
Qwen2.5	5.9	2.0	8.3	0.0
WizardLM-2	12.1	5.6	14.5	7.7

This strengthens our interpretation of the dependency-level findings. At the same time, the presence of unused imports (especially among third-party libraries) still matters from a dependency-management perspective, because even a small number of unnecessary external dependencies can increase integration efforts, maintenance, and add friction overall. We therefore interpret these findings not as evidence of widespread dependency bloat, but as evidence of a more targeted form of integration friction that arises when external libraries are introduced without clear necessity.

Listing 1: A simplified example of an unused import in a generated snippet.

```
import pymongo
from pymongo.errors import BulkWriteError

try:
    results =
    collection.insert_many(documents,
    ordered=False)
except BulkWriteError as e:
    for error in e.details["writeErrors"]:
        if error["code"] == 11000:  #
    duplicate key error
            pass  # ignore duplicate key
```

C. Library-Level Correctness Related to Ground Truth

In Section VI-B, we showed that the libraries imported by LLMs are, in most cases, actually used in the generated code. We now compare these imports against the accepted Stack Overflow answers to assess how closely model-generated libraries align with the benchmark ground truth. In other words, we want to see how often the Python tasks actually require any libraries, when models introduce libraries even though the ground truth does not, and how closely the libraries they choose align with those used in the ground-truth solution. In this analysis, we use the accepted answer from the corresponding Stack Overflow question as a proxy for the ground truth.

To achieve this, we perform a post-hoc comparison between the imports in each model completion and those in the accepted Stack Overflow answer. Using the same code-block extraction and AST-based import identification as in Section II-B, we extract, for every question, (1) the set of modules imported by the accepted answer and (2) the set of modules imported by each model. For each question–model pair, we compare these two sets and assign one of four labels: *ExactMatch* (the model imports exactly the same modules

as the accepted answer), *Superset* (all ground-truth imports plus additional ones), *Subset* (only a subset of the ground-truth imports), or *Different* (neither set contains the other). For questions where the accepted answer imports at least one library, we also compute precision and recall of the model's import set relative to the accepted answer.

How many StackEval Python tasks actually require libraries? Among the 112 Python questions in our StackEval dataset, only 17 (15%) have accepted answers that import at least one library. The remaining 95 questions are solved without any imports in the ground-truth solution.

How often do models import libraries, even when the ground truth does not? Table VI summarises how frequently models import at least one library in their completions. Across all 112 questions, models import libraries in 30 to 48% of their answers (34–54 questions per model), even though only 17 questions actually use imports in the accepted answer. Focusing on the 95 questions whose accepted answer has no imports, model still introduce at least one library in 28–42% of those questions (27–40 questions per model). In other words, all models sometimes over-import libraries on tasks that the ground-truth solution solves without any dependencies, with Qwen2.5 and WizardLM being the biggest "over-importer" in our experiment. Thus, LLMs can propose libraries even when they are not strictly required by the task.

How closely do models match the ground-truth libraries when libraries are required? Table VII focuses on the 17 questions whose accepted answers import at least one library and compares each model's library set to the ground-truth set. DeepSeek V3 and WizardLM-2 exactly reproduce the ground-truth import set in 52.9% of these questions, while GPT-4o and Qwen2.5 do so in 47.1% of cases. For these four models, the remaining outcomes are split between omitting some ground-truth libraries (*Subset*: 29.4% for DeepSeek V3 and 23.5% for GPT-4o, Qwen2.5, and WizardLM-2), adding extras on top of the ground truth (*Superset*: 5.9% for DeepSeek V3, 17.6% for GPT-4o and Qwen2.5, and 11.8% for WizardLM-2), and 11.8% of questions falling into the *Different* category. Overall, these models fully recover the ground-truth import set in 58.8% of the library-requiring questions for DeepSeek V3 and in 64.7% for GPT-4o, Qwen2.5, and WizardLM-2 (*ExactMatch* + *Superset*), which suggests that when libraries are required, they often identify the appropriate dependencies, even if they sometimes add extra imports.

In contrast, Llama 3.1 and O1 Preview omit the libraries used in the accepted answer more often, with *Subset* rates of 64.7% and 58.8%, respectively. Their exact-match rates are also lower, at 23.5% for Llama 3.1 and 29.4% for O1 Preview, and they fully recover the ground-truth import set in only 35.3% and 29.4% of cases, respectively. Combining this with the over-import behavior on tasks where the ground truth uses no libraries, our analysis suggests that models differ not only in which libraries they propose, but also in when they decide to introduce any libraries at all. While this analysis does not guarantee functional correctness, it provides a concrete approximation of correctness at the dependency level and clarifies how often models over- or under-approximate the libraries used in ground-truth solutions. At the same time, this comparison is based on a relatively small subset of 17 questions whose accepted answers contain imports. We therefore interpret these results as complementary evidence about dependency-level correctness rather than as a basis for broad generalization on their own.

Overall, these findings refine the interpretation of our results by showing that the key issue is not broad failure, but a more specific form of dependency-related friction. Most StackEval Python questions do not require imports in the accepted answer, and when libraries are required, several models often select reasonable ones. At the same time, the results show that models may still introduce libraries when none are needed in the ground-truth solution. Therefore, the practical concern is less about universal model failure and more about the tendency to introduce external dependencies in cases where a dependency-free solution would be enough, thereby creating avoidable integration and maintenance burden.

D. Practical Considerations for LLMs in the Context of Dependencies

Our study provides several insights for both practitioners and researchers who integrate LLMs into software development workflows. Importantly, the value of RQ1 and RQ2 is not that models recommend third-party or popular libraries per se, but that they characterize the dependency profile of LLM-generated code in a conversational coding-assistance setting. In particular, these results make visible the trade-offs associated with the dependencies that models introduce, including their installation burden, maintenance implications, licensing properties, and practical usability.

Taken in isolation, the findings that LLMs often recommend third-party libraries and tend to select popular ones are not necessarily surprising or negative. Their practical value lies in what they reveal about the kinds of dependencies models bring into generated code. Recommending third-party libraries more often than standard-library ones increases the external dependency surface of generated solutions, while recommending popular and mature libraries is generally reassuring but does not guarantee friction-free adoption. These descriptive results therefore provide the context needed to interpret the usability, installation, and dependency-resolution issues discussed later in this section.

Licensing awareness and policy checks. Our analysis in RQ2 shows that most libraries recommended by the models use permissive licenses (91% of license instances, primarily MIT, BSD, and Apache-2.0), which aligns well with typical industrial policies. However, we also found (i) one copyleft-licensed recommendation and (ii) a small but recurring set of imports whose license could not be resolved on PyPI due to aliasing or deprecation (e.g., `sklearn`, `imblearn`). This suggests that organizations should not treat LLM recommendations as automatically policy-compliant. Instead, LLM-integrated tools should surface basic license metadata (e.g., SPDX identifier and category) alongside each suggested library and flag libraries that are missing or ambiguous in package registries. At the project level, teams can maintain an allowlist of acceptable license categories and automatically warn when an LLM suggests a library outside that set.

TABLE VI: Frequency with which models import at least one library in their completions. "Any libs" counts questions (out of 112 StackEval Python questions) where the completion imports at least one library. "Libs when GT has none" counts questions (out of the 95 questions whose accepted answer uses no imports) where the model still imports at least one library.

Model	Any libs (112 questions)		Libs when GT has none (95 questions)	
	Occurrence	Percentage	Occurrence	Percentage
DeepSeek V3	49	43.8	35	36.8
GPT-4o	44	39.3	31	32.6
Llama 3.1	44	39.3	36	37.9
O1 Preview	34	30.4	27	28.4
Qwen2.5	54	48.2	40	42.1
WizardLM-2	50	44.6	35	36.8

TABLE VII: Comparison of imported libraries in model completions vs. accepted Stack Overflow answers for the subset of StackEval Python questions whose ground-truth solution imports at least one library (17 out of 112 questions). Values show the percentage of questions in each category and how often each model fully recovers the ground-truth import set.

Model	ExactMatch	Superset	Subset	Different	Fully recovered
DeepSeek V3	52.9	5.9	29.4	11.8	58.8
GPT-4o	47.1	17.6	23.5	11.8	64.7
Llama 3.1	23.5	11.8	64.7	0.0	35.3
O1 Preview	29.4	0.0	58.8	11.8	29.4
Qwen2.5	47.1	17.6	23.5	11.8	64.7
WizardLM-2	52.9	11.8	23.5	11.8	64.7

Resolving imports, not just syntax. RQ3 shows that 4.6% of distinct imported library names in LLM-generated code do not map directly to an installable package, even though the surrounding code is often plausible. Many of these cases are due to aliases (`sklearn` vs. `scikit-learn`) or module-level imports within larger packages, and only 5 out of 24 alias cases included explicit installation guidance. This means that even when the model "knows" the right library, the burden of translating the import into a working installation command is still placed on the user. For practitioners, this implies that LLM suggestions should be treated as incomplete until the import has been resolved to a concrete package name and successfully installed. For tool builders, a practical direction is to couple every suggested import with (i) an explicit installation command (e.g., `pip install scikit-learn`) and (ii) an automatic check against the target package registry; if the check fails or returns a deprecated package, the interface should prompt the user to confirm or correct the library choice.

Over-importing and dependency surface. Our usage analysis reveals that LLMs not only introduce third-party libraries more often than the Python standard library, but they also tend to *over-import*. First, models frequently import libraries even when the accepted Stack Overflow answer solves the task without any imports: for the 95 StackEval questions whose ground-truth solution uses no libraries, models still import at least one library in 28–42% of completions (Table VI). Second, when we examine whether imported libraries are actually referenced, we find that unused imports are more common for third-party libraries (4–15% of third-party imports, depending on the model) than for standard-library imports (0–6%). In other words, when models add unnecessary code, they are more likely to introduce an unnecessary *external* dependency than an unused standard module. For practitioners, this means that accepting LLM code "as is" can quietly inflate a project's dependency surface with libraries that neither the task nor the ground-truth solution requires. It is therefore important to run basic static checks (e.g., dead-import detection) and to question whether a suggested third-party library is actually needed or whether the task can be solved with existing dependencies or standard-library functionality. For researchers and tool builders, these results point to concrete opportunities for LLM-aware linters and IDE integrations that (i) flag unused or low-value imports and (ii) prefer suggestions that minimize the number and scope of new dependencies (e.g., importing a specific submodule rather than an entire framework).

Security implications of non-installable imports. We did not observe hallucinated package names in our dataset, but we did find non-installable imports such as placeholders (e.g., `some_module`) and alias-based names that resolve to deprecated or empty PyPI entries (e.g., `sklearn`). Prior work has shown that such gaps can be exploited via dependency confusion attacks, where an attacker registers a package under an expected name [47]. In practice, this means that even "harmless" placeholders in LLM-generated code can become attack surfaces if they are later published or installed without verification. For practitioners, a simple but effective guideline is to treat any package that is missing from the official registry, recently created, or maintained by an unknown publisher as suspicious until vetted. For researchers, our taxonomy of unresolved imports provides concrete categories—aliases, internal modules, placeholders—that future security-focused evaluations can target when measuring models' susceptibility to hallucination-driven supply-chain attacks.

More generally, our results should be interpreted as identifying friction points in a baseline conversational workflow, not as evidence that such issues are unavoidable in tool-augmented environments. In agentic settings, some of these problems may be mitigated through package-resolution checks, installation support, static analysis, or execution-based feedback. At the same time, those mitigations depend on the surrounding

system being designed to detect and resolve dependency-related issues correctly. Taken together, these findings suggest that LLMs are most effective when embedded in tooling that exposes dependency-level metadata and performs basic validation, rather than used as stand-alone code generators.

VII. Related Works

In this section, we discuss the related literature divided into two aspects. First, we discuss the works that have focused on dependency management and related challenges. Second, we discuss the works that report on the use of large language models as programming assistants.

A. Dependency Management Challenges

While open source software libraries significantly reduce development time and costs [1]–[3], depending on numerous libraries introduces complexity and potential dependency management challenges [4]–[6]. One such challenge is highlighted by Mujahid et al. [7] who identify library selection as a critical aspect of dependency management. In their work, they surveyed developers from the npm ecosystem to qualitatively understand the characteristics of highly-selected libraries. Their results show that JavaScript developers believe that such libraries are well-documented, popular, and free of vulnerabilities. Building upon this work, our study leverages those same characteristics to categorize libraries used by various LLMs. Hauge et al. [8] show that organizational library selection is an ad-hoc process that often relies on a combination of past experiences, expert advice, and online resources. This is further discussed in the work of Haenni et al. [9] where the authors surveyed developers about their decision-making when selecting a library to integrate into their application. Their findings show that, in general, developers do not apply a clear rationale when selecting libraries. Alternatively, developers opted for libraries that fulfilled the immediate task requirements. In this context, LLMs are increasingly being adopted to assist in programming tasks, but little is known about the characteristics of the libraries included in LLM-generated code. Our work empirically analyzes the types of libraries recommended by several state-of-the-art LLMs and assesses their popularity, maintenance, and licensing properties.

Dependency hell is a concept discussed in several studies [48]–[50] and refers to a situation where a project has an excessive number of dependencies, and managing these dependencies becomes difficult and error-prone. Chen et al. [51] discuss the impact of ”trivial packages,” referring to libraries implementing simple functionalities, on the npm ecosystem. Their survey highlights that developers struggle with the multiple dependencies introduced by these libraries, contributing to dependency hell. For instance, a developer reported the cascading effect of patching a deeply nested dependency, requiring updates throughout the dependency tree. Jafari et al. [42] investigate the relationship between npm library characteristics and the dependency update strategies adopted by its dependents. The authors report that the release status, the number of dependents, and the age of a library are the most important indicators of the dependency update strategy. Raemaekers et al. [52] discuss the risks associated with the usage of third-party libraries. They identify key library attributes that could serve as risk indicators. Notably, they report that more popular libraries may be updated more frequently, which increases the chance for new bugs to get introduced into the codebase. These findings support our claim that LLMs could better support developers by providing critical information about the libraries they recommend, such as maintenance metrics (i.e., the number of dependencies, version update frequency, project age). Such transparency can help developers anticipate and manage potential maintenance challenges associated with the increased complexity of dependencies.

B. LLMs in Software Development Workflows

LLMs are increasingly used by software engineering practitioners to perform various tasks, such as code generation, and have shown the potential to improve developer productivity [25], [26], [53]. However, some studies have raised concerns about the reliability of LLM-generated code. For instance, Zhong et al. [54] report common API misuse patterns found in popular LLMs. The study reveals that in the case of GPT-4, 62% of the generated code contained API misuses. The authors argue that this is particularly problematic given that users of LLM code generation are generally not familiar with the APIs that LLMs generate code for, and cannot tell whether the provided code is correct. Similarly, a Vulcan Cyber article claims that ChatGPT hallucinated in almost 40% of the programming questions it was asked [47].

Our findings diverge from those reported in the aforementioned studies. This discrepancy might be attributed to differences in experimental design. First, Zhong et al. employed ”one-shot” or ”few-shot” approaches, providing either irrelevant or relevant examples alongside the prompt. The Vulcan Cyber article describes a scenario where ChatGPT was instructed to generate code for a specific library and then repeatedly prompted for alternatives, potentially leading to hallucinated libraries. Second, we conducted our study on multiple more recent and capable LLMs and aimed to more closely simulate realistic developer-LLM interactions by issuing a single prompt per question. This approach aligns with findings from Jin et al. [55], whose empirical study reveals that developers request code regeneration in only 3% of conversations with ChatGPT.

While LLMs are making significant advancements in software development, concerns regarding reliability and the potential for hallucinations remain. However, studies also highlight the potential of LLMs to improve developer productivity through functionalities like code completion and search. Ross et al. [25] developed an LLM-based programmer’s assistant and evaluated their system on 42 participants. Their results reveal that participants were mostly positive about the assistant’s potential for improving their productivity. Heitz et al. [26] evaluate and compare the performance of OpenAI’s ChatGPT and Google’s Gemini on programming code. They find that while the premium version of the models offers enhanced performance, their free counterparts remain highly relevant for a wide range of users in the context of software development.

Also, the authors report that these models can significantly accelerate coding tasks and improve productivity, but necessitate a rigorous review, especially if the generated code is used in critical areas. Despite this, there is a gap in understanding how LLMs perform in the context of dependencies.

Our study directly addresses this gap by investigating the effectiveness of LLMs in recommending software libraries. We use real-world developer questions from Stack Overflow to analyze the libraries suggested by several recent LLMs (as of 2025) in a setting that reflects practical usage scenarios. This approach avoids the potential bias of controlled experimental prompts. By evaluating factors like library popularity and maintenance, licensing, and potential dependency challenges, our work aims to inform the development of more robust LLMs as programming assistants, and provide developers with recommendations and considerations when using such tools to streamline their workflows in the context of dependencies.

VIII. Threats to Validity

Internal validity considers the experimenter's bias and errors. A potential threat to the validity of our study is the manual classification of libraries and import failures. To mitigate this, all cases were reviewed collaboratively by the authors, and discrepancies were resolved through discussion. We did not employ multiple independent coders or inter-rater agreement measures (e.g., Cohen's kappa), as the classification relied on factual criteria (e.g., `cv2` is an alias for `opencv-python`), not subjective judgment. While this may still introduce some bias, we reduced the likelihood through validation by multiple authors.

For our import-usage analysis, we mark an import as used if its bound name appears in a read context in the snippet's AST and as unused otherwise. This approach may miss dynamic uses or usages in surrounding code that is not part of the completion, and it may classify some imports as "used" even if they are not exercised at runtime. We therefore interpret dead-import rates as conservative, approximate indicators of redundancy rather than exact measures of semantic necessity.

Moreover, the stochastic nature of LLMs means they can generate different outputs for the same input. To reduce this variability, we used a low temperature value (0.01) to produce near-deterministic completions. Finally, prompt formatting and parameter settings can affect LLM outputs. We used default parameters where applicable (except for temperature) and used the Stack Overflow questions for prompting as-is to mimic typical usage and ensure consistency across models

Our findings are also conditioned on a minimally engineered, single-turn prompting setup. As a result, they should not be interpreted as measuring the full potential of agentic LLM systems that can call tools, execute code, invoke package managers, or refine outputs based on feedback. Such systems may mitigate some of the dependency-resolution issues observed in our study. Our results should therefore be understood as a baseline characterization of library-related behavior in conversational coding assistance, while richer agentic evaluations remain outside the scope of this work.

External validity concerns the generalizability of our findings. One external threat relates to the time gap between the Stack Overflow questions and the current software ecosystem. Some libraries may have changed since the questions were posted. To control this, we used the latest available metadata (as of September 2025) when analyzing library characteristics.

Another potential threat is data leakage, where questions from Stack Overflow may have been seen during model training. To assess this, we repeated our analysis on StackUnseen, a companion dataset to StackEval, designed to reduce overlap with LLM training data. For each model, we compared the distributions of recommended-library characteristics (e.g., stars, forks, dependents, dependencies, SourceRank, age, and version frequency) between StackEval and StackUnseen using MannWhitney U tests with Holm correction for multiple comparisons. Across all models and metrics, we found no statistically significant differences (all Holm-corrected $p \geq 0.05$), which suggests that potential data leakage does not materially bias our library-level conclusions. At the same time, StackUnseen can only reduce, not eliminate, the possibility that models were exposed to overlapping or near-duplicate content from other sources. Our results on StackUnseen should therefore be interpreted as approximate lower bounds on data contamination rather than definitive evidence that models fully generalize to entirely unseen questions.

Another threat to external validity is that StackEval is not a benchmark explicitly constructed around library-selection tasks. In many Stack Overflow questions, selecting a library is only one possible means of solving the broader problem, and in some cases it is auxiliary rather than central to the task. As a result, our findings should not be interpreted as measuring LLMs' general ability to choose the optimal library in all software engineering settings. Rather, they characterize model behavior in a Stack Overflow-style conversational Q&A setting, where libraries may be introduced implicitly as part of a generated solution. More generally, benchmark-based LLM evaluations are subject to concerns such as contamination and benchmark-specific overfitting, which can weaken generalization claims [29], [30]. Although our use of StackUnseen helps mitigate some of these concerns, it does not eliminate them entirely.

Our findings are also conditioned on a specific Stack Overflow-style prompt format in which the original question title and body are passed to the model without additional meta-instructions or examples. Prior work shows that LLM outputs can be sensitive to small prompt variations, and prompt sensitivity indices such as POSIX [56] have been proposed to quantify this effect. In this study, we deliberately fix the prompt protocol (StackEval title+body) to 1) mimic typical, minimally engineered Q&A usage, 2) avoid introducing prompt-engineering degrees of freedom, and 3) remain comparable to Shah et al.'s StackEval setup [28]. As a consequence, we interpret our results as characterizing model behaviour under this realistic baseline prompt rather than claiming prompt invariance. Systematically exploring how alternative prompts (e.g., explicitly requesting installation instructions) or prompt-sensitivity metrics (e.g., POSIX) affect library imports is an important direction for future work, but falls outside the scope of this work.

More broadly, our study focuses on a single-turn conver-

sational assistance setting without external tools or iterative feedback. As a result, our findings should not be interpreted as measuring the full potential of agentic LLM systems that can execute code, call package managers, consult linters, or refine outputs based on oracles. Such systems may reduce some of the dependency-resolution issues observed in our study. Our results therefore provide a baseline view of library-related behavior in minimally scaffolded interactions, while richer agentic evaluations remain an important avenue for future work.

Moreover, our analysis focuses on code generated in response to stand-alone questions, without access to an existing project codebase. In practice, LLMs are also used within a repository context (e.g., to add a new function or test to an existing file), where many imports may simply mirror or extend the project's current dependency set rather than introduce entirely new libraries. Studying this scenario would require repository-level benchmarks or logs, together with an analysis pipeline that distinguishes between reused imports and genuinely new dependencies at the project level. In this work, we deliberately stay away from project-specific context to characterize how models select libraries when they are unconstrained by an existing import graph. Our findings should therefore be interpreted as a baseline for library-selection behaviour in conversational coding assistants, and complementary studies on in-repository usage are left to future work.

Our dataset contains 112 Python questions from the StackEval benchmark. While each question is answered by six models, yielding 672 completions and 87 distinct imported libraries, the number of underlying tasks is still small compared to the full extent of Python usage on the web. In addition, the post-hoc comparison against accepted Stack Overflow answers is limited to the subset of questions whose ground-truth solution contains at least one import (17 out of 112 questions). As such, these results should be interpreted as indicative rather than definitive, and broader benchmark coverage would be needed to support stronger generalization. Moreover, the distribution of imported libraries is skewed towards popular third-party packages. Thus, our library-level statistics are most representative of how models handle widely used packages and less informative for niche libraries that appear rarely or not at all in our sample. As a result our quantitative results (e.g., fraction of imports from popular libraries or prevalence of installation guidance) should be interpreted as indicative of model behavior on this curated benchmark rather than exhaustive characterizations of the entire Python ecosystem, as results may differ in other ecosystems, languages, models, or when using different prompts.

IX. Conclusion

In this paper, we assessed how effectively LLMs recommend and use software libraries during code generation. Our findings show that LLMs are generally capable of generating valid and often appropriate library imports across a diverse set of Python programming questions. Across models, the recommended libraries were often well-established, actively maintained, and permissively licensed. We also found that most imports appearing in generated code were actually used, which suggests that library recommendations are usually meaningful rather than arbitrary. At the same time, our results show that models may still introduce external libraries in cases where the ground-truth solution does not require them, pointing to a more subtle form of dependency-related friction in practice.

This friction becomes particularly visible when the recommended import is not straightforward to translate into a working dependency. A non-trivial portion of recommended imports involved cases where the installable package name differed from the import name (e.g., `cv2` for `opencv-python`), or where module-level imports were used without referencing the parent library. In these situations, models often failed to provide installation guidance, leaving developers to manually resolve dependencies. While technically correct, these omissions nonetheless reduce the practical usability of LLM outputs, especially for novice users or in automated workflows.

From a practical standpoint, developers using LLMs for code generation should remain vigilant about dependency validation and license compliance. While most recommended libraries were permissively licensed, we also observed cases where license information was not straightforward to infer from the model completion alone, as well as one case involving a copyleft license, a more restrictive type of license. Thus, although LLM-generated code may accelerate software development, it can still require post-processing to ensure technical and legal compatibility. Tool builders and IDE designers could help close this gap by integrating dependency resolution checks, license-awareness features, and automatic install-command generation directly into LLM-assisted environments. Such support may become even more important in emerging agentic systems, where models can take a more active role in dependency-related actions within the development workflow.

Future work should explore techniques for making LLM outputs more transparent and execution-aware, such as prompting strategies that elicit installation instructions or LLM pipelines augmented with static or runtime analysis. It would also be valuable to study how these issues manifest in agentic development settings, where models may move beyond suggesting code to taking actions that affect project dependencies. Since our study is limited to Python, it remains an open question whether similar patterns hold across other ecosystems such as JavaScript (npm), Java (Maven), or Rust (Cargo).Ultimately, improving the traceability, trustworthiness, and contextual completeness of LLM-generated code in the context of dependencies is critical for its safe integration into software projects.

References

[1] V. R. Basili, L. C. Briand, and W. L. Melo, "How reuse influences productivity in object-oriented systems," *Communications of the ACM*, vol. 39, no. 10, pp. 104–116, 1996.

[2] A. Decan, T. Mens, and P. Grosjean, "An empirical comparison of dependency network evolution in seven software packaging ecosystems," *Empirical Software Engineering*, vol. 24, no. 1, pp. 381–416, 2019.

[3] E. Murphy-Hill, C. Jaspan, C. Sadowski, D. Shepherd, M. Phillips, C. Winter, A. Knight, E. Smith, and M. Jorde, "What predicts software developers' productivity?" *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 582–594, 2019.

[4] J. Latendresse, S. Mujahid, D. E. Costa, and E. Shihab, "Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE'22)*, 2022.

[5] S. Mujahid, "Effective dependency management for the javascript software ecosystem," Ph.D. dissertation, Concordia University, 2021.

[6] Y. Wang, B. Chen, K. Huang, B. Shi, C. Xu, X. Peng, Y. Wu, and Y. Liu, "An empirical study of usages, updates and risks of third-party libraries in java projects," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 35–45.

[7] S. Mujahid, R. Abdalkareem, and E. Shihab, "What are the characteristics of highly-selected packages? a case study on the npm ecosystem," *Journal of Systems and Software*, vol. 198, p. 111588, 2023.

[8] O. Hauge, T. Osterlie, C.-F. Sorensen, and M. Gerea, "An empirical study on selection of open source software-preliminary results," in *2009 ICSE Workshop on Emerging Trends in Free/Libre/Open Source Software Research and Development*. IEEE, 2009, pp. 42–47.

[9] N. Haenni, M. Lungu, N. Schwarz, and O. Nierstrasz, "Categorizing developer information needs in software ecosystems," in *Proceedings of the 2013 international workshop on ecosystem architectures*, 2013, pp. 1–5.

[10] T. Mens and A. Decan, "An overview and catalogue of dependency challenges in open source software package registries," in *The 23rd Belgium-Netherlands Software Evolution Workshop (BENEVOL 2024)*, 2024.

[11] E. Larios Vargas, M. Aniche, C. Treude, M. Bruntink, and G. Gousios, "Selecting third-party libraries: the practitioners' perspective," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 245–256. [Online]. Available: https://doi.org/10.1145/3368089.3409711

[12] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung, "When and how to make breaking changes: Policies and practices in 18 open source software ecosystems," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–56, 2021.

[13] G. Mezzetti, A. Møller, and M. T. Torp, "Type regression testing to detect breaking changes in node. js libraries," in *32nd european conference on object-oriented programming (ECOOP 2018)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 7–1.

[14] A. Møller and M. T. Torp, "Model-based testing of breaking changes in node. js libraries," in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 409–419.

[15] C. Soto-Valero, N. Harrand, M. Monperrus, and B. Baudry, "A comprehensive study of bloated dependencies in the maven ecosystem," *Empirical Software Engineering*, vol. 26, no. 3, p. 45, 2021.

[16] Y. Cao, L. Chen, W. Ma, Y. Li, Y. Zhou, and L. Wang, "Towards better dependency management: A first look at dependency smells in python projects," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1741–1765, 2022.

[17] A. J. Jafari, D. E. Costa, R. Abdalkareem, E. Shihab, and N. Tsantalis, "Dependency smells in javascript projects," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3790–3807, 2021.

[18] C. Soto-Valero, T. Durieux, N. Harrand, and B. Baudry, "Coverage-based debloating for java bytecode," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 2, pp. 1–34, 2023.

[19] H. Mohayeji, A. Agaronian, E. Constantinou, N. Zannone, and A. Serebrenik, "Investigating the resolution of vulnerable dependencies with dependabot security updates," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 234–246.

[20] M. Alfadel, D. E. Costa, E. Shihab, and M. Mkhallalati, "On the use of dependabot security pull requests," in *2021 IEEE/ACM 18th International conference on mining software repositories (MSR)*. IEEE, 2021, pp. 254–265.

[21] R. He, H. He, Y. Zhang, and M. Zhou, "Automating dependency updates in practice: An exploratory study on github dependabot," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4004–4022, 2023.

[22] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, "Vuln4real: A methodology for counting actually vulnerable dependencies," *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592–1609, 2020.

[23] Z. Jia, S. Li, T. Yu, C. Zeng, E. Xu, X. Liu, J. Wang, and X. Liao, "Depowl: detecting dependency bugs to prevent compatibility failures," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 86–98.

[24] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of ai programming assistants: Successes and challenges," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[25] S. I. Ross, F. Martinez, S. Houde, M. Muller, and J. D. Weisz, "The programmer's assistant: Conversational interaction with a large language model for software development," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*, 2023, pp. 491–514.

[26] L. B. Heitz, J. Chamas, and C. Scherb, "Evaluation of the programming skills of large language models," *arXiv preprint arXiv:2405.14388*, 2024.

[27] "Stack Overflow - where developers learn, share, & build careers." [Online]. Available: https://stackoverflow.com/

[28] N. Shah, Z. Genc, and D. Araci, "Stackeval: Benchmarking llms in coding assistance," *Advances in Neural Information Processing Systems*, vol. 37, pp. 36 976–36 994, 2024.

[29] J. Haimes, C. Wenner, K. Thaman, V. Tashev, C. Neo, E. Kran, and J. Schreiber, "Benchmark inflation: Revealing llm performance gaps using retro-holdouts, 2024," *URL https://arxiv. org/abs/2410.09247*.

[30] A. Matton, T. Sherborne, D. Aumiller, E. Tommasone, M. Alizadeh, J. He, R. Ma, M. Voisin, E. Gilsenan-McMahon, and M. Gallé, "On leakage of code generation evaluation datasets," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 13 215–13 223.

[31] A. S. Saabith, T. Vinothraj, and M. Fareez, "Popular python libraries and their application domains," *International Journal of Advance Engineering and Research Development*, vol. 7, no. 11, 2020.

[32] K. Srinath, "Python–the fastest growing programming language," *International Research Journal of Engineering and Technology*, vol. 4, no. 12, pp. 354–357, 2017.

[33] V. Adamson and J. Bägerfeldt, "Assessing the effectiveness of chatgpt in generating python code," 2023.

[34] M. E. Ellis, K. M. Casey, and G. Hill, "Chatgpt and python programming homework," *Decision Sciences Journal of Innovative Education*, vol. 22, no. 2, pp. 74–87, 2024.

[35] T. Zhong, Z. Liu, Y. Pan, Y. Zhang, Y. Zhou, S. Liang, Z. Wu, Y. Lyu, P. Shu, X. Yu *et al.*, "Evaluation of openai o1: Opportunities and challenges of agi," *arXiv preprint arXiv:2409.18486*, 2024.

[36] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[37] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[38] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[39] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang, "Wizardlm: Empowering large language models to follow complex instructions," *arXiv preprint arXiv:2304.12244*, 2023.

[40] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[41] M. Chouchen, N. Bessghaier, M. Begoug, A. Ouni, E. Alomar, and M. W. Mkaouer, "How do software developers use chatgpt? an exploratory study on github pull requests," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 212–216.

[42] A. Javan Jafari, D. E. Costa, E. Shihab, and R. Abdalkareem, "Dependency update strategies and package characteristics," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, sep 2023. [Online]. Available: https://doi.org/10.1145/3603110

[43] R. Cox, "Surviving software dependencies," *Communications of the ACM*, vol. 62, no. 9, pp. 36–43, 2019.

[44] I. Pashchenko, D.-L. Vu, and F. Massacci, "A qualitative study of dependency management and its security implications," in *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, 2020, pp. 1513–1531.

[45] Y. Cheon *et al.*, "Llms as code generators for model-driven development," in *Proceedings of the 20th International Conference on Software Technologies (ICSOFT)*, 2025.

[46] L. Twist *et al.*, "A study of LLMs' preferences for libraries and programming languages," *arXiv preprint arXiv:2503.17181*, 2025.

[47] B. Lanyado, "Can you trust ChatGPT's package recommendations?" 3 2023. [Online]. Available: https://vulcan.io/blog/ai-hallucinations-package-risk

[48] G. Fan, C. Wang, R. Wu, X. Xiao, Q. Shi, and C. Zhang, "Escaping dependency hell: finding build dependency errors with the unified dependency graph," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 463–474.

[49] Y. Tanabe, T. Aotani, and H. Masuhara, "A context-oriented programming approach to dependency hell," in *Proceedings of the 10th ACM International Workshop on Context-Oriented Programming: Advanced Modularity for Run-time Composition*, 2018, pp. 8–14.

[50] R. Abdalkareem, V. Oda, S. Mujahid, and E. Shihab, "On the impact of using trivial packages: An empirical case study on npm and pypi," *Empirical Software Engineering*, vol. 25, pp. 1168–1204, 2020.

[51] X. Chen, R. Abdalkareem, S. Mujahid, E. Shihab, and X. Xia, "Helping or not helping? why and how trivial packages impact the npm ecosystem," *Empirical Software Engineering*, vol. 26, pp. 1–24, 2021.

[52] S. Raemaekers, A. van Deursen, and J. Visser, "Exploring risks in the usage of third-party libraries," in *of the BElgian-NEtherlands software eVOLution seminar*, vol. 31, 2011.

[53] T. Teubner, C. M. Flath, C. Weinhardt, W. van der Aalst, and O. Hinz, "Welcome to the era of chatgpt et al. the prospects of large language models," *Business & Information Systems Engineering*, vol. 65, no. 2, pp. 95–101, 2023.

[54] L. Zhong and Z. Wang, "Can chatgpt replace stackoverflow? a study on robustness and reliability of large language model code generation," 2024.

[55] K. Jin, C.-Y. Wang, H. V. Pham, and H. Hemmati, "Can chatgpt support developers? an empirical evaluation of large language models for code generation," *arXiv preprint arXiv:2402.11702*, 2024.

[56] A. Chatterjee, H. K. Renduchintala, S. Bhatia, and T. Chakraborty, "Posix: A prompt sensitivity index for large language models," *arXiv preprint arXiv:2410.02185*, 2024.